

lecture_15

September 5, 2026

1 Lecture 15: Unconstrained Optimization — Quasi-Newton's Method

{note} Lecture 14 showed that Newton's Method achieves fast quadratic convergence by using the Hessian to reorient the search direction according to local curvature - but computing and inverting the Hessian at every iteration is expensive. This lecture introduces Quasi-Newton methods, which build up curvature information incrementally from gradient observations, achieving near-Newton performance without a single second derivative.

Learning Objectives

By the end of this notebook, you will be able to: 1. Apply first- and second-order optimality conditions to classify stationary points of a smooth unconstrained objective function as local minima, local maxima, or saddle points. 2. Trace one iteration of the BFGS update by hand and explain the role of the secant condition. 3. Explain the trade-offs in convergence speed and computational cost across Gradient Descent, Newton's Method, and BFGS.

Prerequisites: NLP Principles (Lecture 12); Gradient Descent (Lecture 13); Newton's Method (Lecture 14); basic linear algebra (matrix inverse, eigenvalues).

Estimated time: 90 minutes (including in-class exercises)

1.1 Optimality Conditions

1.1.1 First-Order Necessary Condition

If x^* is a local optimum of a differentiable function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, then the gradient:

$$\nabla f(x^*) = \mathbf{0}$$

A point satisfying $\nabla f(x^*) = \mathbf{0}$ is called a stationary point. This is a necessary condition, not sufficient — a stationary point can be a minimum, a maximum, or a saddle point.

1.1.2 Second-Order Conditions

The Hessian $\nabla^2 f(x^*)$ — the $n \times n$ matrix of second partial derivatives — tells us the curvature of f at the stationary point and resolves the ambiguity:

Hessian $\nabla^2 f(x^*)$	All eigenvalues	Character of x^*
Positive definite (PD)	> 0	Local minimum
Negative definite (ND)	< 0	Local maximum
Indefinite	Mixed signs	Saddle point

1.1.3 Example

Delhivery is designing a Pune cross-dock facility. The daily operating cost depends on the number of dock doors x and the sorting line speed y (relative units). The cost function is:

$$C(x, y) = 2x^2 + xy + y^2 - 12x - 10y + 30$$

Setting $\nabla C = \mathbf{0}$:

$$\frac{\partial C}{\partial x} = 4x + y - 12 = 0$$

$$\frac{\partial C}{\partial y} = x + 2y - 10 = 0$$

Solving: from the second equation, $x = 10 - 2y$. Substituting: $4(10 - 2y) + y = 12 \implies y^* = 4$, $x^* = 2$.

The Hessian is:

$$\nabla^2 C = \begin{bmatrix} 4 & 1 \\ 1 & 2 \end{bmatrix}$$

with $\det(\nabla^2 C) = 8 - 1 = 7 > 0$ and $\text{tr}(\nabla^2 C) = 6 > 0$, so both eigenvalues are positive. The Hessian is positive definite and $(x^*, y^*) = (2, 4)$ is a local minimum — the cost-minimizing facility configuration.

1.2 Iterative Descent Framework

For a general nonlinear function f , the system of equations $\nabla f(x) = \mathbf{0}$ may be nonlinear with no closed-form solution. To this end, to find the point where the gradient vanishes for such problems, we start from an initial guess $x^{(0)}$ and repeatedly compute a direction $d^{(k)}$ that moves downhill, then take a step along it $x^{(k+1)} = x^{(k)} + \alpha^{(k)} d^{(k)}$, until the gradient is (approximately) zero. The three methods covered in Lectures 13–15 differ only in how they choose $d^{(k)}$:

Method	Direction $d^{(k)}$	Information used	Convergence
Gradient Descent	$-\nabla f(x^{(k)})$	First-order	Linear

Method	Direction $d^{(k)}$	Information used	Convergence
Newton's Method	$-\nabla^2 f(x^{(k)})^{-1} \nabla f(x^{(k)})$	Second-order	Quadratic
Quasi-Newton's Method (BFGS)	$-\nabla^2 f(x^{(k)})^{-1} \nabla f(x^{(k)}), H^{(k)} \approx \nabla^2 f$	Accumulated first-order	Superlinear

`{tip}` In practice: solve $\nabla f(x) = \mathbf{0}$ analytically if a closed-form solution can be established; otherwise, default to Quasi-Newton's Method; use Newton's Method when the Hessian is cheap to compute and invert; use Gradient Descent when problem scale makes storing a Hessian prohibitive.

1.3 3. Quasi-Newton's Method (BFGS)

While Newton's Method is fast, it demands the full Hessian $\nabla^2 f(x^{(k)})$ at every iteration — analytically and numerically expensive for large problems. Quasi-Newton methods retain the curvature-aware structure of the Newton direction while replacing the true Hessian with a matrix $H^{(k)}$ built entirely from first-order information accumulated across iterations.

The Broyden–Fletcher–Goldfarb–Shanno (BFGS) algorithm is the most widely used quasi-Newton method. It initializes $H^{(0)} = \mathbf{I}$ (making the first step identical to Gradient Descent) and refines $H^{(k)}$ after each iteration using the secant condition — the requirement that the updated approximate Hessian reproduce the observed gradient change over the last step, exactly as the true Hessian would via the mean-value theorem.

Defining the step displacement and gradient change:

$$s^{(k)} = x^{(k+1)} - x^{(k)}, \quad y^{(k)} = \nabla f(x^{(k+1)}) - \nabla f(x^{(k)})$$

the secant condition requires $H^{(k+1)} s^{(k)} = y^{(k)}$. The BFGS rank-2 update satisfies this condition while keeping $H^{(k+1)}$ symmetric and positive definite:

$$H^{(k+1)} = H^{(k)} - \frac{H^{(k)} s^{(k)} (s^{(k)})^\top H^{(k)}}{(s^{(k)})^\top H^{(k)} s^{(k)}} + \frac{y^{(k)} (y^{(k)})^\top}{(y^{(k)})^\top s^{(k)}}$$

The BFGS search direction and update are then:

$$d^{(k)} = -[H^{(k)}]^{-1} \nabla f(x^{(k)}), \quad x^{(k+1)} = x^{(k)} + \alpha^{(k)} d^{(k)}$$

where $\alpha^{(k)}$ is chosen by a line search satisfying the Wolfe conditions.

Early in the algorithm, when $H^{(k)} \approx \mathbf{I}$, the search direction closely resembles Gradient Descent. As iterations accumulate, $H^{(k)}$ progressively refines its estimate of the true Hessian and the direction increasingly resembles the Newton direction — without ever computing a single second derivative.

BFGS converges superlinearly in practice — faster than linear convergence, approaching quadratic near the optimum — at a fraction of the cost of Newton’s Method.

1.4 In-Class Exercises

1.4.1 Exercise 1

An NHAI corridor study models total system travel time across two parallel links — the Mumbai–Pune Expressway (v_1) and old NH48 (v_2), with volumes in thousands of vehicles/hour — as:

$$T(v_1, v_2) = 3v_1^2 - 2v_1v_2 + 2v_2^2 - 14v_1 - 12v_2 + 50$$

1. Find the stationary point analytically and classify it using the Hessian.
2. Trace 1 iteration of BFGS by hand, starting from $(v_1, v_2) = (0, 0)$ with $H^{(0)} = \mathbf{I}$ and exact line search. Fill in the table below and compute $H^{(1)}$ using the BFGS update formula. Compare BFGS with the result from Newton’s Method (Lecture 14) and Gradient Descent (Lecture 13).

1.4.2 Exercise 2

Delhivery is minimizing operational cost at its Chennai cross-dock facility. The daily cost as a function of throughput λ (tonnes/day) and staffing level s (workers) is:

$$C(\lambda, s) = 2\lambda^2 - 4\lambda s + 3s^2 - 12\lambda - 6s + 50$$

1. Find the stationary point analytically and classify it using the Hessian.
2. Trace 1 iteration of BFGS by hand, starting from $(\lambda, s) = (0, 0)$ with $H^{(0)} = \mathbf{I}$ and exact line search. Fill in the table below and compute $H^{(1)}$ using the BFGS update formula. Compare BFGS with the result from Newton’s Method (Lecture 14) and Gradient Descent (Lecture 13).
3. What does $H^{(1)}$ “know” that $H^{(0)} = \mathbf{I}$ did not?

BFGS Table

Quantity	Value
$x^{(0)}$	$(0, 0)$
$\nabla f(x^{(0)})$	
$d^{(0)} = -[H^{(0)}]^{-1}\nabla f(x^{(0)})$	
$\alpha^{(0)}$ (exact line search)	
$x^{(1)}$	
$\nabla f(x^{(1)})$	
$s^{(0)} = x^{(1)} - x^{(0)}$	
$y^{(0)} = \nabla f(x^{(1)}) - \nabla f(x^{(0)})$	
$H^{(1)}$	

1.5 Take-Away Exercises

1.5.1 Exercise 1

BMTC is optimizing its demand-responsive feeder service in Bengaluru. The total daily operating cost depends on fleet size n (vehicles) and headway h (minutes). The cost model is:

$$C(n, h) = 2n^2 + 3h^2 + 2nh - 20n - 30h + 100$$

where the terms capture vehicle capital cost, passenger waiting cost, and a joint fleet-frequency interaction respectively.

1. Find the stationary point analytically and classify it using the Hessian.
2. Trace 1 iteration of BFGS by hand, starting from $(n, h) = (0, 0)$ with $H^{(0)} = \mathbf{I}$ and exact line search. Fill in the table below and compute $H^{(1)}$ using the BFGS update formula. Compare BFGS with the result from Newton's Method (Lecture 14) and Gradient Descent (Lecture 13).

Quantity	Value
$x^{(0)}$	$(0, 0)$
$\nabla f(x^{(0)})$	
$d^{(0)} = -[H^{(0)}]^{-1} \nabla f(x^{(0)})$	
$\alpha^{(0)}$ (exact line search)	
$x^{(1)}$	
$\nabla f(x^{(1)})$	
$s^{(0)} = x^{(1)} - x^{(0)}$	
$y^{(0)} = \nabla f(x^{(1)}) - \nabla f(x^{(0)})$	
$H^{(1)}$	

1.5.2 Exercise 2

An NHAI planning study models the full-lifecycle cost of a four-lane corridor as a function of traffic volume v (thousand veh/day) and maintenance frequency m (interventions/year):

$$F(v, m) = 3v^2 - 2vm + 2m^2 - 18v - 12m + 80$$

1. Find the stationary point analytically and classify it using the Hessian.
2. Implement BFGS in Python using `scipy.optimize.minimize` with `method='BFGS'`, starting from $(v, m) = (0, 0)$. Report the number of function evaluations and iterations. Compare BFGS with the result from Newton's Method (Lecture 14) and Gradient Descent (Lecture 13).

1.6 Further Reading

- Nocedal, J. and Wright, S.J. (2006). *Numerical Optimization* (2nd ed.). Springer — Chapter 6 (quasi-Newton methods, BFGS derivation and convergence).

- Bazaraa, M.S., Sherali, H.D., and Shetty, C.M. (2006). *Nonlinear Programming: Theory and Algorithms* (3rd ed.). Wiley — Chapter 9 (Newton and quasi-Newton methods).
- SciPy documentation: `scipy.optimize.minimize` — [docs.scipy.org](https://docs.scipy.org/doc/scipy/reference/optimize.html)
- BPR (1964). *Traffic Assignment Manual*. US Bureau of Public Roads, Washington DC.